Lesson 3: Vector Databases & Retrieval

Now that you understand embeddings, let's learn how to store and search through millions of them efficiently.

The Challenge: Scaling Semantic Search

Remember our simple search from Lesson 2?

```
# This works for 5 documents...
similarities = cosine_similarity(query_embedding, doc_embeddings)
```

But what about 1 million documents? - Computing 1M similarities per query is slow - Storing 1M × 1536-dimensional vectors = ~6GB of memory - We need a better solution!

Enter: Vector Databases

What is a Vector Database?

A **vector database** is a specialized database optimized for: 1. **Storing** high-dimensional vectors (embeddings) 2. **Indexing** vectors for fast retrieval 3. **Searching** for similar vectors efficiently

Think of it as a search engine for embeddings.

Vector DB vs. Traditional DB

Traditional Database	Vector Database

Stores structured data (rows/columns)	Stores vectors (embeddings)
Searches for exact matches	Searches for similar vectors
Uses indexes (B-trees, hash tables)	Uses vector indexes (ANN)
Query: `WHERE name = 'John'`	Query: `SIMILAR TO vector`

How Vector Databases Work

1. Approximate Nearest Neighbor (ANN) Search

Instead of comparing with every vector (slow), vector databases use **ANN algorithms**:

Exact Search (Naive):

```
# Compare with ALL vectors - O(n)
for doc_embedding in all_embeddings:
   similarity = cosine_similarity(query, doc_embedding)
# Time: Linear with number of documents ■
```

Approximate Search (ANN):

```
# Use an index to find APPROXIMATE nearest neighbors - O(log n)
similar_vectors = vector_index.search(query, top_k=5)
# Time: Logarithmic or constant! ■
# Trade-off: Might miss some relevant results (but very close)
```

2. Popular ANN Algorithms

Different vector databases use different algorithms:

HNSW (Hierarchical Navigable Small World)

- **Used by**: Chroma, Weaviate, Qdrant
- **How it works**: Creates a multi-layer graph structure
- **Pros**: Very fast queries, high accuracy
- **Cons**: High memory usage, slower indexing

```
Layer 2:    A ←→ B
            ↓    ↓
Layer 1:    A ←→ C ←→ B ←→ D
            ↓    ↓    ↓    ↓
Layer 0:    A-C-E-F-B-G-H-D-I-J
```

IVF (Inverted File Index)

- **Used by**: Faiss, Milvus
- **How it works**: Clusters vectors into regions
- **Pros**: Memory efficient, good for large datasets
- **Cons**: Lower accuracy than HNSW

```
Cluster 1: [vectors about cars]
Cluster 2: [vectors about food]
Cluster 3: [vectors about sports]
→ Search only relevant clusters!
```

Product Quantization

- **Used by**: Faiss, Pinecone
- **How it works**: Compresses vectors to save memory
- **Pros**: Very memory efficient
- **Cons**: Some quality loss

Popular Vector Databases

---

1. ChromaDB

Best for: Getting started, local development

```
import chromadb

# Create client
client = chromadb.Client()

# Create collection
collection = client.create_collection("my_docs")

# Add documents
collection.add(
   documents=["Paris is the capital of France", "Python is a language"],
   ids=["doc1", "doc2"],
   metadatas=[{"source": "geography"}, {"source": "tech"}]
)

# Query
results = collection.query(
   query_texts=["What is the capital of France?"],
   n_results=2
)
```

Pros: Simple, no setup, good for learning
Cons: Not for large-scale production

2. Pinecone

Best for: Production, serverless, managed

```
import pinecone

# Initialize
pinecone.init(api_key="your-key")
index = pinecone.Index("my-index")

# Upsert vectors
index.upsert([
   ("doc1", [0.1, 0.2, ...], {"text": "Paris is..."}),
   ("doc2", [0.3, 0.4, ...], {"text": "Python is..."})
])

# Query
results = index.query(
   vector=[0.15, 0.22, ...],
   top_k=2,
   include_metadata=True
)
```

Pros: Fully managed, scales automatically, fast
Cons: Paid service, vendor lock-in

3. Weaviate

Best for: Complex queries, hybrid search

```
import weaviate

client = weaviate.Client("http://localhost:8080")

# Define schema
schema = {
   "class": "Document",
```

```
    "properties": [{"name": "content", "dataType": ["text"]}]
}
client.schema.create_class(schema)

# Add data
client.data_object.create(
    {"content": "Paris is the capital of France"},
    "Document"
)

# Query
result = client.query.get("Document", ["content"]).with_near_text({
    "concepts": ["capital of France"]
}).do()
```

Pros: Feature-rich, GraphQL API, hybrid search

Cons: More complex setup

4. Faiss (Facebook AI)

Best for: Research, high performance, local

```
import faiss
import numpy as np

# Create index
dimension = 384
index = faiss.IndexFlatL2(dimension)  # L2 distance

# Add vectors
vectors = np.random.random((1000, dimension)).astype('float32')
index.add(vectors)

# Search
query = np.random.random((1, dimension)).astype('float32')
distances, indices = index.search(query, k=5)
```

Pros: Extremely fast, many algorithms, free

Cons: No built-in metadata, lower-level API

Comparison Table

Database	Best For	Ease of Use	Scale	Cost
ChromaDB	Learning, prototypes	■■■■■	Small-Medium	Free
Pinecone	Production, serverless	■■■■	Large	Paid
Weaviate	Complex queries	■■■	Medium-Large	Free/Paid
Faiss	Research, on-prem	■■	Very Large	Free
Qdrant	Self-hosted production	■■■■	Large	Free/Paid
Milvus	Enterprise	■■■	Very Large	Free/Paid

Document Chunking

Before storing documents, you need to split them into chunks.

Why Chunk?

1. **Token limits**: Embedding models have max input length
2. **Retrieval precision**: Smaller chunks = more precise retrieval
3. **Context windows**: LLMs have limited context length

Chunking Strategies

1. Fixed-Size Chunking

Split by number of characters/tokens with overlap:

```
def chunk_text(text, chunk_size=500, overlap=50):
    """Split text into overlapping chunks"""
    chunks = []
    start = 0
    while start < len(text):
        end = start + chunk_size
        chunks.append(text[start:end])
        start = end - overlap  # Overlap for context
    return chunks

text = "Long document..." * 100
chunks = chunk_text(text, chunk_size=500, overlap=50)
```

Pros: Simple, predictable

Cons: May split mid-sentence or mid-thought

2. Sentence-Based Chunking

Split on sentence boundaries:

```
import nltk
nltk.download('punkt')

def chunk_by_sentences(text, sentences_per_chunk=5):
    """Split text into chunks of N sentences"""
    sentences = nltk.sent_tokenize(text)
    chunks = []
    for i in range(0, len(sentences), sentences_per_chunk):
        chunk = ' '.join(sentences[i:i+sentences_per_chunk])
        chunks.append(chunk)
    return chunks
```

Pros: Maintains semantic boundaries

Cons: Variable chunk sizes

3. Paragraph-Based Chunking

Split on paragraphs (double newlines):

```
def chunk_by_paragraphs(text, max_size=1000):
    """Split text by paragraphs, combining small ones"""
    paragraphs = text.split('\n\n')
    chunks = []
    current_chunk = ""

    for para in paragraphs:
        if len(current_chunk) + len(para) < max_size:
            current_chunk += para + "\n\n"
        else:
            if current_chunk:
                chunks.append(current_chunk.strip())
            current_chunk = para + "\n\n"

    if current_chunk:
        chunks.append(current_chunk.strip())

    return chunks
```

Pros: Respects document structure

Cons: Uneven chunk sizes

4. Semantic Chunking (Advanced)

Split based on topic changes using embeddings:

```
from sentence_transformers import SentenceTransformer
import numpy as np

def semantic_chunking(text, similarity_threshold=0.7):
    """Split when topic changes (detected by embedding similarity)"""
    sentences = nltk.sent_tokenize(text)
    model = SentenceTransformer('all-MiniLM-L6-v2')
    embeddings = model.encode(sentences)

    chunks = []
    current_chunk = [sentences[0]]

    for i in range(1, len(sentences)):
        # Compare with previous sentence
        similarity = cosine_similarity(
```

```
            embeddings[i-1:i],
            embeddings[i:i+1]
        )[0][0]

        if similarity > similarity_threshold:
            current_chunk.append(sentences[i])
        else:
            # Topic changed, start new chunk
            chunks.append(' '.join(current_chunk))
            current_chunk = [sentences[i]]

    chunks.append(' '.join(current_chunk))
    return chunks
```

Pros: Respects semantic boundaries
Cons: Computationally expensive

Chunking Best Practices

1. **Overlap chunks**: Prevents losing context at boundaries
2. **Include metadata**: Store source, page number, section
3. **Test different sizes**: Optimal size depends on your use case
4. **Balance**: Too small = loss of context, too large = irrelevant info

Recommended starting point: 500-1000 characters with 10-20% overlap

Metadata and Filtering

Store metadata with your embeddings for filtering:

```
# Add documents with metadata
collection.add(
    documents=["Paris is the capital...", "Python is a language..."],
    metadatas=[
        {"source": "geography.pdf", "page": 12, "date": "2024-01-01"},
        {"source": "programming.pdf", "page": 5, "date": "2024-02-15"}
    ],
    ids=["doc1", "doc2"]
)

# Query with filters
results = collection.query(
    query_texts=["programming languages"],
    n_results=5,
    where={"source": "programming.pdf"}  # Only search in programming docs!
)
```

Common metadata fields: - `source`: Original document/file - `page`: Page number - `section`: Chapter or section - `date`: Creation/modification date - `author`: Document author - `category`: Document category/type

Retrieval Strategies

---

1. Top-K Retrieval (Most Common)

```
# Return top 3 most similar documents
results = collection.query(
    query_texts=["What is the capital of France?"],
    n_results=3
)
```

Best for: Most use cases

Trade-off: May include irrelevant results if not enough relevant docs

2. Similarity Threshold

```
# Return only documents above similarity threshold
results = collection.query(
    query_texts=["What is the capital of France?"],
    n_results=10
)
```

```
# Filter by threshold
filtered_results = [
    r for r in results
    if r['distance'] < 0.5  # Lower distance = higher similarity
]
```

Best for: Ensuring quality, avoiding irrelevant results

Trade-off: Might return zero results

3. MMR (Maximum Marginal Relevance)

Balance relevance with diversity:

```
# Return diverse results (not all similar to each other)
results = collection.query(
    query_texts=["machine learning"],
    n_results=5,
    search_type="mmr",  # Maximum Marginal Relevance
    lambda_mult=0.5     # 0 = diverse, 1 = similar
)
```

Best for: Avoiding redundant results

Example: Don't return 5 documents all about Python when query is "programming"

4. Hybrid Search

Combine semantic search with keyword search:

```
# Semantic search
semantic_results = vector_search(query_embedding)

# Keyword search
keyword_results = fulltext_search(query_text)

# Combine results (various strategies)
final_results = combine_and_rerank(semantic_results, keyword_results)
```

Best for: Handling specific terms (names, IDs, codes)
Example: "GPT-4" is a specific term that benefits from keyword matching

Hands-On: Building a Vector DB RAG System

Let's build a complete example with ChromaDB:

```
import chromadb
from sentence_transformers import SentenceTransformer
from typing import List

class SimpleRAG:
    def __init__(self, collection_name="docs"):
        # Initialize ChromaDB
        self.client = chromadb.Client()
        self.collection = self.client.create_collection(collection_name)

        # Initialize embedding model
        self.model = SentenceTransformer('all-MiniLM-L6-v2')

    def add_documents(self, documents: List[str], metadatas: List[dict] = None):
        """Add documents to the vector database"""
        # Generate IDs
        ids = [f"doc_{i}" for i in range(len(documents))]

        # Add to collection (ChromaDB handles embedding)
        self.collection.add(
            documents=documents,
            ids=ids,
            metadatas=metadatas
        )
        print(f"Added {len(documents)} documents")

    def search(self, query: str, n_results: int = 3):
        """Search for relevant documents"""
        results = self.collection.query(
            query_texts=[query],
            n_results=n_results
        )

        return {
            'documents': results['documents'][0],
            'metadatas': results['metadatas'][0],
            'distances': results['distances'][0]
        }

# Usage
rag = SimpleRAG()

# Add documents
documents = [
```

```
    "Paris is the capital and largest city of France.",
    "The Eiffel Tower was built in 1889 and is located in Paris.",
    "Python is a high-level programming language.",
    "Machine learning is a subset of artificial intelligence.",
    "The Louvre Museum in Paris is the world's largest art museum.",
]

metadatas = [
    {"category": "geography"},
    {"category": "landmarks"},
    {"category": "programming"},
    {"category": "ai"},
    {"category": "culture"},
]

rag.add_documents(documents, metadatas)

# Search
results = rag.search("What can you tell me about Paris?", n_results=3)

print("\nSearch Results:")
for i, (doc, dist) in enumerate(zip(results['documents'], results['distances'])):
    print(f"{i+1}. (distance: {dist:.3f}) {doc}")
```

Output:

```
Added 5 documents

Search Results:
1. (distance: 0.432) Paris is the capital and largest city of France.
2. (distance: 0.621) The Eiffel Tower was built in 1889 and is located in Paris.
3. (distance: 0.678) The Louvre Museum in Paris is the world's largest art museum.
```

Performance Optimization

---

1. Batch Operations

```
# Bad: One at a time
for doc in documents:
    collection.add(documents=[doc], ids=[f"doc_{i}"])

# Good: Batch insert
collection.add(documents=documents, ids=ids)
```

2. Index Tuning

```
# Faiss example: Choose right index type
import faiss

# For < 1M vectors: Flat index (exact search)
index = faiss.IndexFlatL2(dimension)

# For > 1M vectors: IVF index (approximate)
quantizer = faiss.IndexFlatL2(dimension)
index = faiss.IndexIVFFlat(quantizer, dimension, 100)  # 100 clusters
```

3. Caching

```
# Cache frequently used queries
from functools import lru_cache

@lru_cache(maxsize=1000)
def cached_search(query: str):
   return collection.query(query_texts=[query])
```

Common Issues and Solutions

Issue 1: Slow Queries

Causes: Too many vectors, inefficient index
Solutions: - Use ANN algorithms (HNSW, IVF) - Reduce dimensionality (PCA) - Add metadata filters to narrow search - Use better hardware (more RAM)

Issue 2: Poor Retrieval Quality

Causes: Bad chunking, wrong embedding model, not enough context
Solutions: - Experiment with chunk sizes - Try different embedding models - Add metadata for filtering - Use hybrid search - Implement re-ranking

Issue 3: Out of Memory

Causes: Too many high-dimensional vectors
Solutions: - Use product quantization - Store vectors on disk (with index in memory) - Use managed service (Pinecone, Weaviate) - Reduce dimensionality

What You've Learned

■ Vector databases enable fast similarity search at scale
■ ANN algorithms trade slight accuracy for massive speed gains
■ Chunking strategies affect retrieval quality
■ Metadata enables filtering and better context
■ Different retrieval strategies serve different use cases

Practice Exercises

1. **Build your own**: Create a RAG system with ChromaDB using your own documents

2. **Compare chunking**: Try different chunk sizes and measure retrieval quality

3. **Add metadata**: Implement filtering by document source or date

4. **Benchmark**: Compare query times with 100 vs. 10,000 documents

Next Steps

In Lesson 4: Language Models & Generation, you'll learn: - How LLMs work at a high level - Effective prompting techniques - Integrating retrieved context with generation - Handling edge cases

Further Reading

- ChromaDB Documentation
- Pinecone Learning Center
- HNSW Algorithm Paper
- Faiss Documentation

← Lesson 2: Understanding Embeddings | Next: Language Models & Generation →